

Introduction to Data Science

Programming II: Iteration, automation, scheduling

Simon Munzert

Hertie School | GRAD-C11/E1339

Table of contents

1. Iteration
2. Automation and scripting
3. Scheduling

Iteration

Iteration



Iteration

The ubiquity of iteration

- Often we have to run the same task over and over again, with minor variations. Examples:
 - Standardize values of a variable
 - Recode all numeric variables in a dataset
 - Running multiple models with varying covariate sets
- A benefit of scripting languages in data (as opposed to point-and-click solutions) is that we can easily automate the process of iteration

Ways to iterate

- A simple approach is to copy-and-paste code with minor modifications (→ "duplicate code", → "copy-and-paste programming"). This is lazy, error-prone, not very efficient, and violates the "Don't repeat yourself" (DRY) principle.
- In R, **vectorization**, that is applying a function to every element of a vector at once, already does a good share of iteration for us.
- `for()` **loops** are intuitive and straightforward to build, but involve repetitive bookkeeping (more on that in a minute).
- Finally, we learned about functions. Now, we learn how to unleash their power by applying them to anything we interact with in R at scale.

A simple example

Task

Say we want to double each element in a numeric vector,

`x = c(1, 2, 3, 4, 5)`. Here are some different approaches to achieve this:

1. Manually (sometimes: copying and pasting code)

```
R> x <- c(1, 2, 3, 4, 5)
R> x_doubled <- c(2, 4, 6, 8, 10)
```

2. Vectorization

```
R> x <- c(1, 2, 3, 4, 5)
R> x_doubled <- x * 2
R> x_doubled
```

```
[1] 2 4 6 8 10
```

3. `for()` loop

```
R> x <- c(1, 2, 3, 4, 5)
R> x_doubled <- numeric(length(x))
R> for (i in seq_along(x)) {
+   x_doubled[i] <- x[i] * 2
+ }
R> x_doubled
```

```
[1] 2 4 6 8 10
```

4. Using `purrr`

```
R> library(purrr)
R> x <- c(1, 2, 3, 4, 5)
R> x_doubled <- map_dbl(x, ~ .x * 2)
R> x_doubled
```

```
[1] 2 4 6 8 10
```

Iteration with for() loops

Anatomy of a for() loop

Every `for()` loop has three components:

1. **Output:** a container for the results, allocated *before* the loop starts.
2. **Sequence:** what to loop over. `seq_along(x)` generates one index per element of `x`.
3. **Body:** the code that does the actual work, run once per iteration.

Example

Compute the median of every column of a data frame:

```
R> df <- starwars %>%  
+   select(height, mass, birth_year)  
R>  
R> medians <- numeric(ncol(df))      # output  
R> for (i in seq_along(df)) {        # sequence  
+   medians[i] <- median(df[[i]],    # body  
+                           na.rm = TRUE)  
+ }  
R> medians
```

```
[1] 180  79  52
```

Iteration with for() loops: good practice

Two classic rookie mistakes

1. **Growing the output.** `c(medians, ...)` copies the entire vector in every iteration — fine for 10 elements, painfully slow for 100,000. Always pre-allocate.
2. **`1:length(x)` as sequence.** If `x` happens to have length zero, `1:0` yields the sequence `c(1, 0)` — the loop runs twice on an empty object and fails cryptically. `seq_along(x)` handles this edge case correctly.

How not to do it

```
R> medians <- c()                                # no! grows...
R> for (i in 1:length(df)) {                      # no! 1:0 bug
+   medians <- c(medians,
+               median(df[[i]], na.rm = TRUE))
+ }
```

Related: while() loops

When you don't know the number of iterations in advance (e.g., "sample until we have 50 valid cases"), use a `while()` loop:

```
R> while (n_valid < 50) {
+   # sample, validate, increment n_valid
+ }
```

From loops to functionals

for() loops...

- ... are **not slow** — that reputation is outdated; poorly written loops (see previous slide) are slow.
- ... are maximally explicit: every step is visible, no new vocabulary needed.
- ... are the right tool when iterations **depend on each other**, e.g., simulations where step t needs the result of step $t-1$.
- **But:** the interesting part (the body) is buried in bookkeeping — output, sequence, index.
- **And:** the bookkeeping is repetitive and error-prone. It's the same for every loop — so why write it again and again?

Functionals (map() & friends)...

- ... wrap the loop machinery for you: allocation, sequencing, and collection happen inside the function.
- ... express intent: `map_dbl(df, median)` says *what* happens, not *how*.
- ... eliminate the index bugs from the previous slide by design.
- ... are just loops under the hood — someone else wrote the boilerplate (in fast C code, by the way).

"Of course someone has to write loops. It doesn't have to be you." — Jenny Bryan

Iteration with purrr

The tidyverse way to iterate

- For *real* functional programming in base R, we can use the `*apply()` family of functions (`lapply()`, `sapply()`, etc.). See [here](#) for an excellent summary.
- In the tidyverse, this functionality comes with the `purrr` package.
- At its core is the `map*()` family of functions.

How `purrr` works

- The idea is always to **apply** a function to **x**, where x can be a list, vector, data.frame, or something more complex.
- The output is then returned as output of a pre-defined type (e.g., a list).



Iteration with purrr: map()

The `map*()` functions all follow a similar syntax:

$$\textit{map}(.x, .f, \dots)$$

We use it to apply a function `.f` to each piece in `.x`. Additional arguments to `.f` can be passed on in `...`.

For instance, if we want to identify the object class of every column of a data.frame, we can write:

```
R> map(starwars, class)
```

```
$name  
[1] "character"
```

```
$height  
[1] "integer"
```

```
$mass  
[1] "numeric"
```

```
$hair_color  
[1] "character"
```

```
$skin_color  
[1] "character"
```

Iteration with purrr: map() cont.

By default, `map()` returns a list. But we can also use other `map*()` functions to give us an atomic vector of an indicated type (e.g., `map_int()` to return an integer vector, or `map_vec()` to return a vector that is the simplest common type).

Going back to the previous example, we can also use `map_chr()`, which returns a character vector:

```
R> map_chr(starwars, class)
```

name	height	mass	hair_color	skin_color	eye_color
"character"	"integer"	"numeric"	"character"	"character"	"character"
birth_year	sex	gender	homeworld	species	films
"numeric"	"character"	"character"	"character"	"character"	"list"
vehicles	starships				
"list"	"list"				

The `purrr` function set is quite comprehensive. Be sure to check out the [cheat sheet](#) and the [tutorials](#). You'll survive without `purrr` but you probably don't want to live without it. Together with `dplyr` it's easily the most powerful package for data wrangling in the tidyverse. If you master it, it will save you a lot of time and headaches.

Iteration with purrr: map() cont.

Apply functions with purrr : : CHEATSHEET

Map Functions

ONE LIST

map(x, f, ...) Apply a function to each element of a list or vector, and return a list.
`x <- list(a = 1:10, b = 11:20, c = 21:30)`
`l1 <- list(x = c("a", "b"), y = c("c", "d"))`
`map(l1, sort, decreasing = TRUE)`



map_dbl(x, f, ...)
Return a double vector.
`map_dbl(x, mean)`



map_int(x, f, ...)
Return an integer vector.
`map_int(x, length)`



map_chr(x, f, ...)
Return a character vector.
`map_chr(l1, paste, collapse = "")`



map_lgl(x, f, ...)
Return a logical vector.
`map_lgl(x, is.integer)`



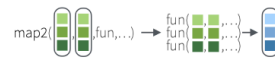
map_vec(x, f, ...)
Return a vector that is of the simplest common type.
`map_vec(l1, paste, collapse = "")`



walk(x, f, ...) Trigger side effects, return invisibly.
`walk(x, print)`

TWO LISTS

map2(x, y, f, ...) Apply a function to pairs of elements from two lists or vectors, return a list.
`y <- list(1, 2, 3); z <- list(4, 5, 6); l2 <- list(x = "a", y = "z")`
`map2(x, y, \(x, y) x * y)`



map2_dbl(x, y, f, ...) Return a double vector.
`map2_dbl(y, z, ~ x / y)`



map2_int(x, y, f, ...) Return an integer vector.
`map2_int(y, z, ~ +)`



map2_chr(x, y, f, ...) Return a character vector.
`map2_chr(l1, l2, paste, collapse = "", sep = ";")`



map2_lgl(x, y, f, ...) Return a logical vector.
`map2_lgl(l2, l1, ~ %in% `)`



map2_vec(x, y, f, ...) Return a vector that is of the simplest common type.
`map2_vec(l1, l2, paste, collapse = "", sep = ";")`



walk2(x, y, f, ...) Trigger side effects, return invisibly.
`walk2(objs, paths, save)`

imap(x, f, ...) is shorthand for **map2(x, names(x), f)** or **map2(x, seq_along(x), f)** depending on whether **x** is named or not.

MANY LISTS

pmap(l, f, ...) Apply a function to groups of elements from a list of lists or vectors, return a list.
`pmap(list(x, y, z), function(first, second, third) first * (second + third))`



pmap_dbl(l, f, ...) Return a double vector.
`pmap_dbl(list(y, z), ~ x / y)`



pmap_int(l, f, ...) Return an integer vector.
`pmap_int(list(y, z), ~ +)`



pmap_chr(l, f, ...) Return a character vector.
`pmap_chr(list(l1, l2), paste, collapse = "", sep = ";")`



pmap_lgl(l, f, ...) Return a logical vector.
`pmap_lgl(list(l2, l1), ~ %in% `)`



pmap_vec(l, f, ...) Return a vector that is of the simplest common type.
`pmap_vec(list(l1, l2), paste, collapse = "", sep = ";")`



pwalk(l, f, ...) Trigger side effects, return invisibly.
`pwalk(list(objs, paths), save)`

Function Shortcuts

Use **\(x)** with functions like **map()** that have single arguments.

`map(l, \(x) x + 2)`
becomes
`map(l, function(x) x + 2)`

Use **\(x, y)** with functions like **map2()** that have two arguments.

`map2(l, p, \(x, y) x + y)`
becomes
`map2(l, p, function(l, p) l + p)`

Use **\(x, y, z)** etc with functions like **pmap()** that have many arguments.

`pmap(list(x, y, z), \(x, y, z) x + y / z)`
becomes
`pmap(list(x, y, z), function(x, y, z) x * (y + z))`

Use **\(x, y)** with functions like **imap()**. **x** will get the list value and **y** will get the index, or name if available.

`imap(list("a", "b", "c"), \(x, y) paste0(y, ":", x))`
outputs "index: value" for each item

Use a **string** or an **integer** with any map function to index list elements by name or position. `map(l, "name")` becomes `map(l, function(x) x[["name"]])`



Iteration with purrr: map() cont.

Modify



modify(x, .f, ...) Apply a function to each element. Also **modify2()**, and **imodify()**.
`modify(x, ~.+2)`



modify_at(x, .at, .f, ...) Apply a function to selected elements. Also **map_at()**.
`modify_at(x, "b", ~.+2)`



modify_if(x, .p, .f, ...) Apply a function to elements that pass a test. Also **map_if()**.
`modify_if(x, is.numeric, ~.+2)`



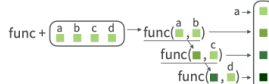
modify_depth(x, .depth, .f, ...) Apply function to each element at a given level of a list. Also **map_depth()**.
`modify_depth(x, 1, ~.+2)`

Reduce

reduce(x, .f, ..., .init, .dir = c("forward", "backward"))
Apply function recursively to each element of a list or vector. Also **reduce2()**.
`reduce(x, sum)`



accumulate(x, .f, ..., .init) Reduce a list, but also return intermediate results. Also **accumulate2()**.
`accumulate(x, sum)`



Vectors



compact(x, .p = identity)
Discard empty elements.
`compact(x)`



keep_at(x, .at)
Keep/discard elements based by name or position. Conversely, **discard_at()**.
`keep_at(x, "a")`



set_names(x, .nm = x)
Set the names of a vector/list directly or with a function.
`set_names(x, c("p", "q", "r"))`
`set_names(x, tolower)`

Predicate functions



keep(x, .p, ...)
Keep elements that pass a logical test. Conversely, **discard()**.
`keep(x, is.numeric)`



head_while(x, .p, ...)
Return head elements until one does not pass. Also **tail_while()**.
`head_while(x, is.character)`



detect(x, .f, ..., .dir = c("forward", "backward"), .right = NULL, .default = NULL)
Find first element to pass.
`detect(x, is.character)`



detect_index(x, .f, ..., .dir = c("forward", "backward"), .right = NULL)
Find index of first element to pass.
`detect_index(x, is.character)`



every(x, .p, ...)
Do all elements pass a test?
`every(x, is.character)`



some(x, .p, ...)
Do some elements pass a test?
`some(x, is.character)`



none(x, .p, ...)
Do no elements pass a test?
`none(x, is.character)`



has_element(x, .y)
Does a list contain an element?
`has_element(x, "foo")`

Pluck



pluck(x, ..., .default=NULL)
Select an element by name or index. Also **attr_getter()** and **chunk()**.
`pluck(x, "b")`
`x |> pluck("b")`



assign_in(x, .where, .value)
Assign a value to a location using pluck selection.
`assign_in(x, "b", 5)`
`x |> assign_in("b", 5)`



modify_in(x, .where, .f)
Apply a function to a value at a selected location.
`modify_in(x, "b", abs)`
`x |> modify_in("b", abs)`

Reshape



list_flatten(x)
Remove a level of indexes from a list.
`list_flatten(x)`

List-Columns

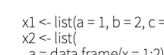


List-columns are columns of a data frame where each element is a list or vector instead of an atomic value. Columns can also be lists of data frames. See **tidyr** for more about nested data and list columns.

WORK WITH LIST-COLUMNS

Manipulate list-columns like any other kind of column, using **dplyr** functions like **mutate()**. Because each element is a list, use **map** functions within a column function to manipulate each element.

Concatenate



`x1 <- list(a = 1, b = 2, c = 3)`
`x2 <- list(a = data.frame(x = 1:2), b = data.frame(y = "a"))`
`list_c(x1, x2)`



list_c(x) Combines elements into a vector by concatenating them together.
`list_c(x1)`



list_rbind(x) Combines elements into a data frame by row-binding them together.
`list_rbind(x2)`



list_cbind(x) Combines elements into a data frame by column-binding them together.
`list_cbind(x2)`



list_ranspose(.l, .names = NULL)
Transposes the index order in a multi-level list.
`list_transpose(x)`

map(), map2(), or pmap() return lists and will create new list-columns.

`starwars |> transmute(ships = map2(vehicles, starships, append))`

Suffixed map functions like **map_int()** return an atomic data type and will **simplify list-columns into regular columns**.

`starwars |> mutate(n_films = map_int(films, length))`

Iteration with purrr: specifying the function

Three ways to specify .f

```
R> # a named function
R> map_dbl(mtcars, mean)
```

mpg	cyl	disp	hp	drat
20.090625	6.187500	230.721875	146.687500	3.596563
vs	am	gear	carb	
0.437500	0.406250	3.687500	2.812500	

```
R> # an anonymous function (base R, >= 4.1)
R> map_dbl(mtcars, \(x) length(unique(x)))
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
25	3	27	22	22	29	30	2	2	3	6

```
R> # a purrr-style formula ("lambda")
R> map_dbl(mtcars, ~ length(unique(.x)))
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
25	3	27	22	22	29	30	2	2	3	6

Bonus: extracting elements

Instead of a function, you can pass a name or position to extract elements from nested lists:

```
R> fellowship <- list(
+   list(name = "Frodo", race = "Hobbit", age = 51),
+   list(name = "Gandalf", race = "Maia", age = 2019)
+   list(name = "Legolas", race = "Elf", age = 2931)
+ )
R> map_chr(fellowship, "name") # by name
```

```
[1] "Frodo" "Gandalf" "Legolas"
```

```
R> map_dbl(fellowship, 3) # by position
```

```
[1] 51 2019 2931
```

This is extremely handy for deeply nested data, e.g., parsed JSON from web APIs (more on that soon!).

Iteration with purrr: a common use case

Reading many files at once

- A classic data science scenario: your data live in many files of the same structure — one per year, country, survey wave, ...
- Instead of writing one `read_csv()` call per file, let `map()` do the work:

```
R> paths <- list.files("data/polls",  
+                     pattern = "\\*.csv$",  
+                     full.names = TRUE)  
R> polls_df <- paths %>%  
+   map(read_csv) %>% # list of data frames  
+   list_rbind()      # bind into one data frame
```

- `map(paths, read_csv)` is just shorthand for `list(read_csv(paths[[1]]), read_csv(paths[[2]]), ...)`.

More than one input

- `map2(.x, .y, .f)` iterates over two vectors in parallel; `pmap(.l, .f)` generalizes to any number of arguments.
- The `walk()` family works like `map()` but is called for its side effects (saving, printing) and returns no output:

```
R> # save one plot per group  
R> walk2(plots, paths,  
+       \(plot, path) ggsave(path, plot))
```



Iteration with purrr: dealing with failure

One bad apple...

When you apply a function to many inputs, a single failure kills the entire operation:

```
R> x <- list(4, 16, "sixty-four", 256)
R> map_dbl(x, log2)
```

```
Error in `map_dbl()`:  
i In index: 3.  
Caused by error in `.f()`:  
! non-numeric argument to mathematical function
```

Annoying with 4 elements — painful when input 3,912 of 4,000 fails after an hour of scraping or model fitting, and all completed results are lost.

...doesn't have to spoil the barrel

`possibly()` takes a function and returns a modified version that never throws an error but returns a default value instead:

```
R> map_dbl(x, possibly(log2, otherwise = NA_real_))  
  
[1] 2 4 NA 8
```

Back to the many-files example — read what can be read, skip the rest:

```
R> paths %>%  
+   map(possibly(read_csv, otherwise = NULL)) %>%  
+   list_rbind() # NULLs are dropped automatically
```

Also check out the siblings `safely()` (returns result and error) and `quietly()` (captures messages and warnings).

Iterating over columns: across()

The problem

Applying the same function to many columns is one of the most common iteration tasks. Here's an example doing it the copy-and-paste way:

```
R> summarise(  
+   height_mean = mean(height, na.rm = TRUE),  
+   height_sd   = sd(height, na.rm = TRUE),  
+   mass_mean   = mean(mass, na.rm = TRUE),  
+   mass_sd     = sd(mass, na.rm = TRUE)  
+ )
```

The same pattern, repeated four times — that's a violation of the DRY principle. Imagine doing this for 20 variables...

The solution: across()

`dplyr`'s `across()` applies one or more functions to a selection of columns inside `summarize()` or `mutate()`:

```
R> starwars %>%  
+   group_by(species) %>%  
+   summarize(  
+     across(c(height, mass),  
+           list(mean = \(x) mean(x, na.rm = TRUE),  
+                 sd   = \(x) sd(x, na.rm = TRUE)))  
+   print(n = 5)
```

```
# A tibble: 38 × 5  
  species height_mean height_sd mass_mean mass_sd  
  <chr>      <dbl>      <dbl>      <dbl>      <dbl>  
1 Aleena          79         NA          15         NA  
2 Besalisk       198         NA         102         NA  
3 Cerean         198         NA          82         NA  
4 Chagrian       196         NA         NaN         NA  
5 Clawdite       168         NA          55         NA  
# i 33 more rows
```

Iterating over columns: across() cont.

Selecting columns flexibly

The first argument of `across()` uses the same tidy-select syntax you know from `select()`, including helpers:

```
R> # all numeric columns
R> starwars %>%
+   summarize(across(where(is.numeric),
+                        \ (x) median(x, na.rm = TRUE)))
R>
R> # all (non-grouping) columns
R> starwars %>%
+   summarize(across(everything(), n_distinct))
R>
R> # turn all character columns into factors
R> starwars %>%
+   mutate(across(where(is.character), as.factor))
```

The `.names` argument controls the naming of output columns, e.g. `.names = "{.fn}_{.col}"`.

Related: if_any() and if_all()

`across()` has two useful cousins for `filter()`: `if_any()` and `if_all()` combine column-wise logical checks into a single condition.

```
R> # rows with missings in height OR mass
R> starwars %>%
+   filter(if_any(c(height, mass), is.na)) %>%
+   select(name, height, mass) %>%
+   print(n = 5)
```

```
# A tibble: 28 × 3
  name          height  mass
  <chr>         <int> <dbl>
1 Wilhuff Tarkin    180    NA
2 Mon Mothma       150    NA
3 Arvel Crynyd      NA     NA
4 Finis Valorum    170    NA
5 Rugor Nass       206    NA
# i 23 more rows
```

Iteration in a nutshell: the map family

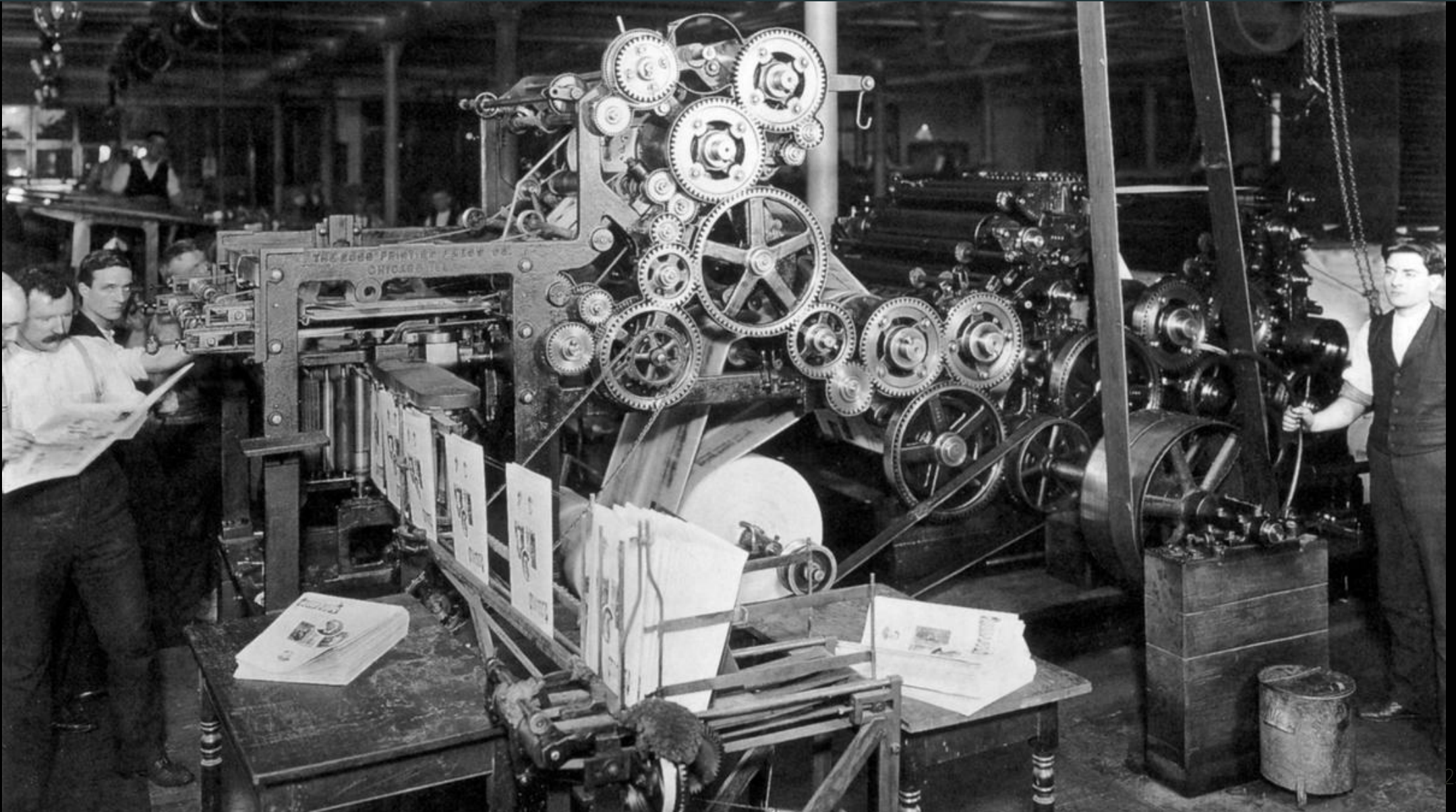
Function	What it does	Think of it as...
<code>map(x, f)</code>	Applies <code>f</code> to every element of <code>x</code> ; returns a list	A conveyor belt: every item passes through the same machine; the outputs are collected in separate boxes
<code>map_dbl()</code> , <code>map_chr()</code> , <code>map_lgl()</code> , <code>map_int()</code>	Like <code>map()</code> , but returns an atomic vector of the promised type	The same conveyor belt with strict quality control at the exit: only one type of product leaves the factory
<code>map2(x, y, f)</code>	Iterates over two vectors in parallel	A zipper: pairs up the elements of two vectors, one pair per iteration
<code>pmap(l, f)</code>	Iterates over any number of arguments simultaneously	A cook working through a recipe table: each row holds the ingredients for one dish
<code>walk(x, f)</code> , <code>walk2()</code>	Like <code>map()</code> / <code>map2()</code> , but called for side effects ; returns nothing	A mail carrier: delivers the letters (saves files, prints output) and comes back empty-handed

Iteration in a nutshell: helpers and friends

Function	What it does	Think of it as...
<code>possibly(f, otherwise)</code>	Wraps <code>f</code> ; on error returns a default value instead of failing	An airbag: a crash doesn't end the journey — you get a placeholder and keep driving
<code>safely(f)</code>	Wraps <code>f</code> ; always returns both result and error	A flight recorder: keeps the result <i>and</i> a record of what went wrong for later inspection
<code>quietly(f)</code>	Wraps <code>f</code> ; captures messages, warnings, and printed output	A court stenographer: nothing is said out loud, but everything is on record
<code>list_rbind()</code>	Binds a list of data frames into one data frame	A stapler: stacks the pages into a single document
<code>across(cols, f)</code>	Applies <code>f</code> to a selection of columns inside <code>summarize()</code> / <code>mutate()</code> (honorary member from dplyr)	A car wash: every selected column gets the same treatment

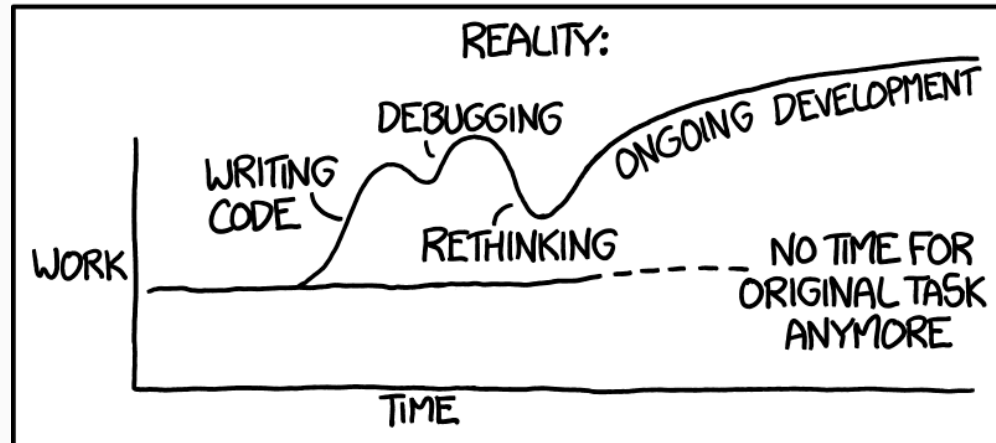
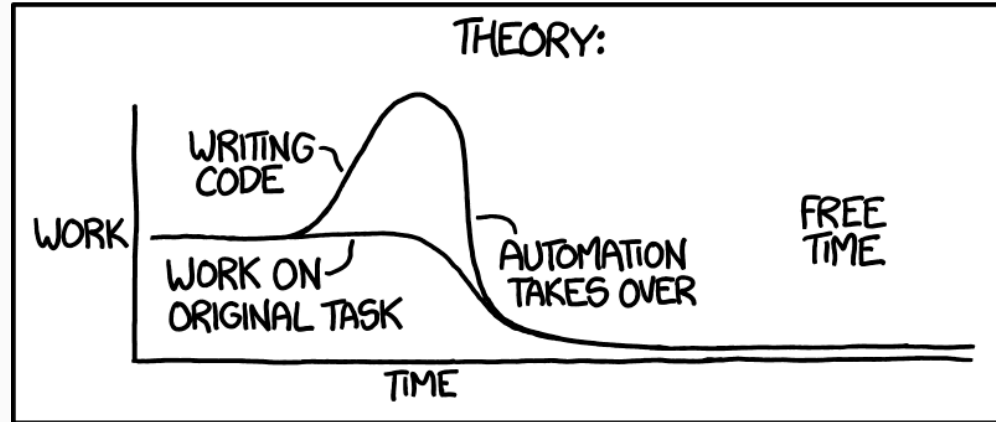
Automation and scripting

Automation



Automation

"I SPEND A LOT OF TIME ON THIS TASK.
I SHOULD WRITE A PROGRAM AUTOMATING IT!"



Credit [Randall Munroe/xkcd 1319](#)

Automation

Motivation

- We spend **too much time** on repetitive tasks.
- We're already automating using scripts that bundle multiple commands! Next step: The pipeline as a series of scripts and commands.
- Good pipelines are modular. But you don't want to trigger 10 scripts sequentially by hand.
- Some tasks are to be repeated on a regular basis (schedule).

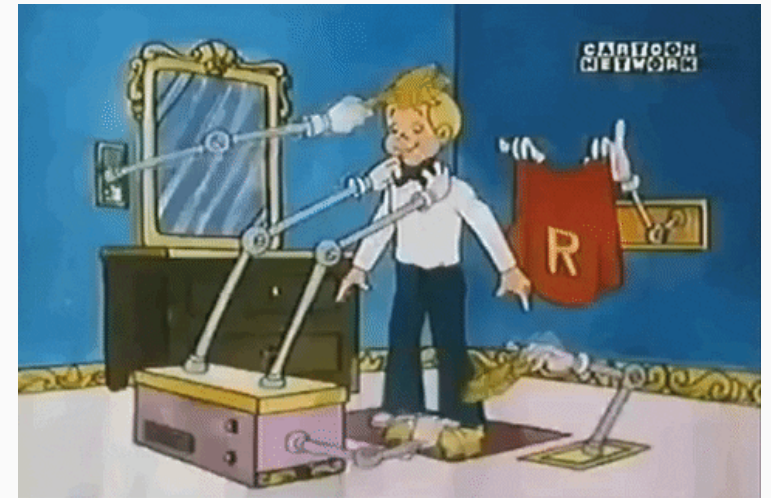
When automation makes sense

- The input is variable but the process of turning input into output is highly standardized.
- You use a diverse set of software to produce the output.
- Others (humans, machines) are supposed to run the analyses.
- Time saved by automation >> Time needed to automate.

Different ways of doing it

We will consider automation

- using **R**,
- using the **Shell** and **RScript**,
- using **make**, and
- using dedicated **scheduling tools**.



Thinking in pipelines

Key characteristics

- Pipelines make complex projects easier to handle because they break up a monolithic script into **discrete, manageable chunks**.
- If properly done, each stage of the pipeline defines its input and its outputs.
- Pipeline modules **do not modify their inputs** (*idempotence*). Rerunning one module produces the same results as the previous run.

Key advantages

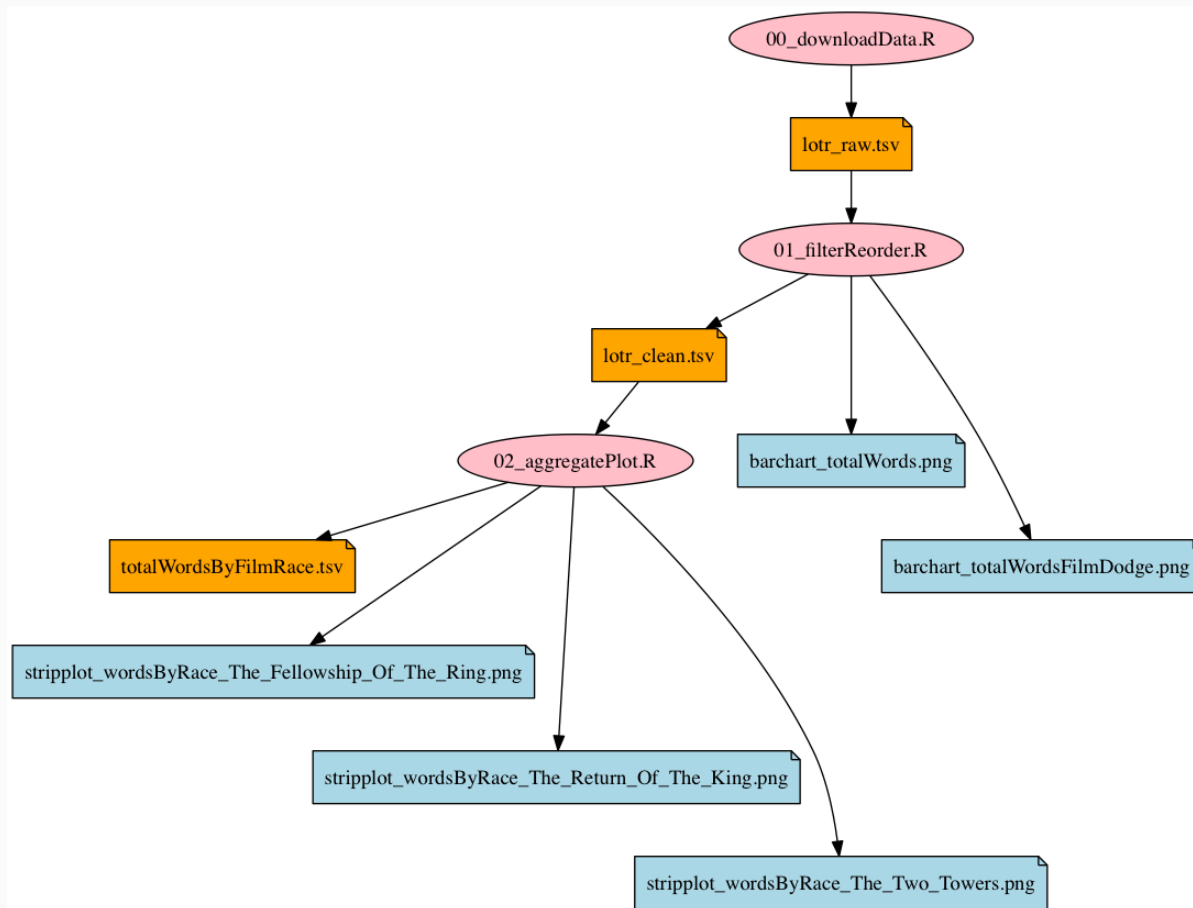
- When you modify one stage of the pipeline, you only have to rerun the downstream, dependent stages.
- Division of labor is straightforward.
- Modules tend to be a lot easier to debug.



A data science pipeline is a graph

Wait what

- Scripts and data files are vertices of the graph.
- Dependencies between stages are edges of the graph.
- Pipelines are not necessarily DAGs. Recursive routines are imaginable (but to be avoided?).
- Also, scripts are not necessarily hierarchical (e.g., multiple different modeling approaches of the same data in different scripts).
- An automation script gives *one* order in which you can successfully run the pipeline.



An example pipeline

In the following, we will work with this toy pipeline:¹

¹Courtesy of Jenny Bryan.

An example pipeline

In the following, we will work with this toy pipeline:

- `00-packages.R` loads the packages necessary for analysis,

`00-packages.R`:

```
R> # install packages from CRAN
R> p_needed <- c("tidyverse" # tidyverse packages
+ )
R> packages <- rownames(installed.packages())
R> p_to_install <- p_needed[!(p_needed %in% packages)]
R> if (length(p_to_install) > 0) {
+   install.packages(p_to_install)
+ }
R> lapply(p_needed, require, character.only = TRUE)
```

An example pipeline

In the following, we will work with this toy pipeline:

- `00-packages.R` loads the packages necessary for analysis,
- `01-download-data.R` downloads a spreadsheet, which is stored as `lotr_raw.tsv`,

`01-download-data.R`:

```
R> ## download raw data
R> download.file(url = "http://bit.ly/lotr_raw-tsv",
+               destfile = "lotr_raw.tsv")
```

An example pipeline

In the following, we will work with this toy pipeline:

- `00-packages.R` loads the packages necessary for analysis,
- `01-download-data.R` downloads a spreadsheet, which is stored as `lotr_raw.tsv`,
- `02-process-data.R` imports and processes the data and exports a clean spreadsheet as `lotr_clean.tsv`, and

`02-process-data.R`:

```
R> ## import raw data
R> lotr_dat <- read_tsv("lotr_raw.tsv")
R>
R> ## reorder Film factor levels based on story
R> old_levels <- levels(as.factor(lotr_dat$Film))
R> j_order <- sapply(c("Fellowship", "Towers", "Return"),
+                   function(x) grep(x, old_levels))
R> new_levels <- old_levels[j_order]
R>
R> ## process data set
R> lotr_dat <- lotr_dat %>%
+   # apply new factor levels to Film
+   mutate(Film = factor(as.character(Film), new_levels),
+   # revalue Race
+   Race = recode(Race, `Ainur` = "Wizard", `Men` = "Man")) %>%
+ ## <skipping some steps here to avoid slide overflow>
+
+ ## write data to file
+ write_tsv(lotr_dat, "lotr_clean.tsv")
```

An example pipeline

In the following, we will work with this toy pipeline:

- `00-packages.R` loads the packages necessary for analysis,
- `01-download-data.R` downloads a spreadsheet, which is stored as `lotr_raw.tsv`,
- `02-process-data.R` imports and processes the data and exports a clean spreadsheet as `lotr_clean.tsv`, and
- `03-plot.R` imports the clean dataset, produces a figure and exports it as `barchart-words-by-race.png`.

`03-plot.R`:

```
R> ## import clean data
R> lotr_dat <- read_tsv("lotr_clean.tsv") %>%
+ # reorder Race based on words spoken
+ mutate(Race = reorder(Race, Words, sum))
R>
R> ## make a plot
R> p <- ggplot(lotr_dat, aes(x = Race, weight = Words)) + geom_bar()
R> ggsave("barchart-words-by-race.png", p)
```

An example pipeline

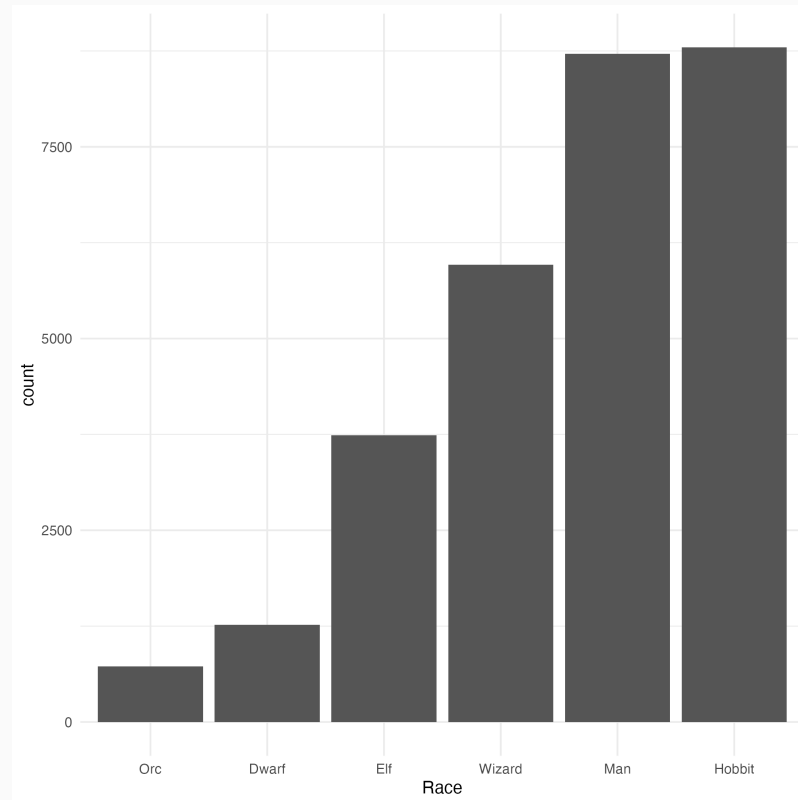
```
R> slice_sample(lotr_dat, n = 10)
```

```
# A tibble: 10 × 5
```

	Film	Chapter	Character	Race	Words
	<chr>	<chr>	<chr>	<chr>	<dbl>
1	The Return Of The King	64: The Mouth Of Sauron	Aragorn	Man	23
2	The Fellowship Of The Ring	36: The Bridge Of Khazad-dûm	Frodo	Hobb...	4
3	The Two Towers	36: Isengard Unleashed	Saruman	Wiza...	50
4	The Fellowship Of The Ring	42: The Great River	Sam	Hobb...	37
5	The Return Of The King	42: Breaking The Gate Of Go...	Gandalf	Wiza...	21
6	The Two Towers	45: The Glittering Caves	Legolas	Elf	36
7	The Two Towers	35: Helm's Deep	Rohan Wa...	Man	22
8	The Fellowship Of The Ring	33: Moria	Aragorn	Man	31
9	The Fellowship Of The Ring	43: Parth Galen	Aragorn	Man	79
10	The Return Of The King	24: Courage Is The Best Def...	Gothmog	Orc	4

An example pipeline

```
R> p <- ggplot(lotr_dat, aes(x = Race, weight = Words)) +  
+   geom_bar() + theme_minimal()
```



Automation using pipelines in R

Motivation and usage

- The `source()` function reads and parses R code from a file or connection.
- We can build a pipeline by sourcing scripts sequentially.
- This pipeline is usually stored in a "master/main" script.
- The removal of previous work is optional and maybe redundant. Often the data is overwritten by default.
- It is recommended that the individual scripts are (partial) standalones, i.e. that they import all data they need by default (loading the packages could be considered an exception).
- Note that as long as the environment is not reset, it remains intact across scripts, which is a potential source of error and confusion.

Example

The master script `master.R`:

```
R> ## clean out any previous work
R> outputs <- c("lotr_raw.tsv",
+              "lotr_clean.tsv",
+              list.files(pattern = "*.png$"))
R> file.remove(outputs)
R>
R> ## run scripts
R> source("00-packages.R")
R> source("01-download-data.R")
R> source("02-process-data.R")
R> source("03-plot.R")
```

Automation using the Shell and Rscript

Motivation and usage

- Alternatively to using an R master script, we can also run the pipeline from the command line.
- Note that here, the environments don't carry over across `Rscript` calls. The scripts definitely have to run in a standalone fashion (i.e., load packages, import all necessary data, etc.).
- The working directory should be set either in the script(s) or in the shell with `cd`.

Example

The master script `master.sh`:

```
#!/bin/sh
cd /Users/simonmunzert/github/examples/02-automation
set -eux
Rscript 01-download-data.R
Rscript 02-process-data.R
Rscript 03-plot.R
```

The `set` command allows to adjust some base shell parameters:

- `-e`: Stop at first error
- `-u`: Undefined variables are an error
- `-x`: Print each command as it is run

For more information on `set`, see [here](#).

Automation using the Shell and Rscript

Motivation and usage

- Alternatively to using an R master script, we can also run the pipeline from the command line.
- Note that here, the environments don't carry over across `Rscript` calls. The scripts definitely have to run in a standalone fashion (i.e., load packages, import all necessary data, etc.).
- The working directory should be set either in the script(s) or in the shell with `cd`.
- One advantage of this approach is that it can be easily coupled with other command line tools, building a **polyglot pipeline**.

Example

The master script `master.sh`:

```
#!/bin/sh
cd /Users/simonmunzert/github/examples/02-automation
set -eux
curl -L http://bit.ly/lotr_raw-tsv > lotr_raw.tsv
Rscript 02-process-data.R
Rscript 03-plot.R
```

The `set` command allows to adjust some base shell parameters:

- `-e`: Stop at first error
- `-u`: Undefined variables are an error
- `-x`: Print each command as it is run

For more information on `set`, see [here](#).

Automation using Make

Motivation and usage

- **Make** is an automation tool that allows us to specify and manage build processes.
- It is commonly run via the shell.
- At the heart of a make operation is the `makefile` (or `Makefile`, `GNUmakefile`), a script which serves as a recipe for the building process.
- A `makefile` is written following a particular syntax and in a declarative fashion.
- Conceptually, the recipe describes which files are built how and using what input.

Advantages of Make

- It looks at which files you have and automatically figures out how to create the files that you have. For complex pipelines this "automation of the automation process" can be very helpful.
- While shell scripts give *one* order in which you can successfully run the pipeline, Make will figure out the parts of the pipeline (and their order) that are needed to build a desired target.



Automation using Make (cont.)

Basic syntax

Each batch of lines indicates

- a file to be created (the target),
- the files it depends on (the prerequisites), and
- set of commands needed to construct the target from the dependent files.

Dependencies propagate.

- To create any of the `png` figures, we need `lotr_clean.tsv`.
- If this file changes, the `png`s change as well when they're built.

Example `makefile`

```
all: lotr_clean.tsv barchart-words-by-race.png words-histogram.png

lotr_raw.tsv:
    curl -L http://bit.ly/lotr_raw-tsv > lotr_raw.tsv

lotr_clean.tsv: lotr_raw.tsv 02-process-data.R
    Rscript 02-process-data.R

barchart-words-by-race.png: lotr_clean.tsv 03-plot.R
    Rscript 03-plot.R

words-histogram.png: lotr_clean.tsv
    Rscript -e 'library(ggplot2);
    qplot(Words, data = read.delim("$<"), geom = "histogram");
    ggsave("$@")'
    rm Rplots.pdf

clean:
    rm -f lotr_raw.tsv lotr_clean.tsv *.png
```

Automation using Make (cont.)

Getting Make to run

- Using the command line, go into the directory for your project.
- Create the `Makefile` file.
- The most basic Make commands are `make all` and `make clean` which builds (or deletes) all output as specified in the script.

Example `makefile`

```
all: lotr_clean.tsv barchart-words-by-race.png words-histogram.png

lotr_raw.tsv:
    curl -L http://bit.ly/lotr_raw-tsv > lotr_raw.tsv

lotr_clean.tsv: lotr_raw.tsv 02-process-data.R
    Rscript 02-process-data.R

barchart-words-by-race.png: lotr_clean.tsv 03-plot.R
    Rscript 03-plot.R

words-histogram.png: lotr_clean.tsv
    Rscript -e 'library(ggplot2);
    qplot(Words, data = read.delim("$<"), geom = "histogram");
    ggsave("$@")'
    rm Rplots.pdf

clean:
    rm -f lotr_raw.tsv lotr_clean.tsv *.png
```

Automation using Make - FAQ

Does it work on Windows?

To install and run `make` on Windows, check out [these instructions](#).

Where can I learn more?

If you consider working with Make, check out the [official manual](#), [this helpful tutorial](#), Karl Broman's [excellent minimal make introduction](#), or [this Stat545 piece](#).

This is dusty technology. Are there alternatives?

In the context of data science with R, the `targets` package is an interesting option. It provides R functionality to define a Make-style pipeline. Check out the [overview](#) and [manual](#).



Scheduling

Scheduling



Scheduling

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

Credit [Randall Munroe/xkcd 1205](#)

Scheduling scripts and processes

Motivation

- So far, we have automated data science pipelines.
- But the execution of these pipelines still needs to be triggered.
- In some cases, it is desirable to also **automate the initialization** of R scripts (or any processes for that matter) **on a regular basis**, e.g. weekly, daily, on logon, etc.
- This makes particular sense when you have moving parts in your pipeline (most likely: data).

Common scenarios for scheduling

1. You fetch data from the web on a regular basis (e.g., via scraping scripts or APIs).
2. You generate daily/weekly/monthly reports/tweets based on changing data.
3. You build an alert control system informing you about anomalies in a database.

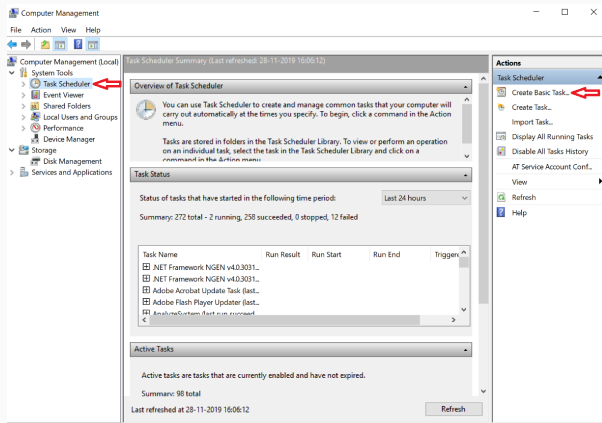


Credit [Simone Giertz](#)

Scheduling scripts and processes on Windows

Scheduling options

- Schedule tasks on Windows with **Windows Task Scheduler**.
- Manage them via a GUI (→ Control Panel) or the command line using `schtasks.exe`.
- The R package `taskschedulerR` provides a programmable R interface to the WTS.



taskschedulerR example

```
R> library(taskschedulerR)
R> myscript <- "examples/scrape-wiki.R"
R> ## Run every 5 minutes, starting from 10:40
R> taskscheduler_create(
+   taskname = "WikiScraperR_5min", rscript = myscript,
+   schedule = "MINUTE", starttime = "10:40", modifier = 5)
R>
R> ## Run every week on Saturday and Sunday at 09:10
R> taskscheduler_create(
+   taskname = "WikiScraperR_SatSun", rscript = myscript,
+   schedule = "WEEKLY", starttime = "09:10",
+   days = c('SAT', 'SUN'))
R>
R> ## Delete task
R> taskscheduler_delete("WikiScraperR_SatSun")
R>
R> ## Get a data.frame of all tasks
R> tasks <- taskscheduler_ls()
R> str(tasks)
```

Scheduling scripts and processes on a Mac

Scheduling options

- On macOS you can schedule background jobs using `cron` and `launchd`.
- `launchd`¹ was created by Apple as a replacement for the popular Linux utility `cron` (deprecated but still usable).
- The R package `cronR` provides a programmable R interface.
- `cron` syntax for more complex scheduling:

1

2

3

4

5

/PATH/TO/FILE

Minute 0-59

Hour 0-23

Day of Month 1-31

Month 1-12

Day of Week 0-6

Timing Syntax Example

0 8 *** This will run once a day at 8AM

0 *** 0 This will run on Sunday every hour

0 10-15 *** This will run once every hour between 10:00 to 15:00

¹For more resources on `launchd`, check out [this](#).

`cronR` example

```
R> library(cronR)
R> myscript <- "examples/scrape-wiki.R"
R> # Create bash code for crontab to execute R script
R> cmd <- cron_rscript(myscript)
R>
R> ## Run every minute
R> cron_add(command = cmd, frequency = 'minutely',
+           id = 'ScraperR_1min', description = 'Every 1min')
R>
R> ## Run every 15 minutes (using cron syntax)
R> cron_add(cmd, frequency = '*/15 * * * *',
+           id = 'ScraperR_15min', description = 'Every 15 mins')
R>
R> ## Check number of running cronR jobs
R> cron_njobs()
R>
R> ## Delete task
R> cron_rm("WikiScraperR_1min", ask = TRUE)
```